ICT4153

Mobile Application Development

Practical 02

Flutter Installation

Objective

To set up the Flutter environment in the desktop.

To set up Android Studio for Flutter development and run a sample Flutter application.

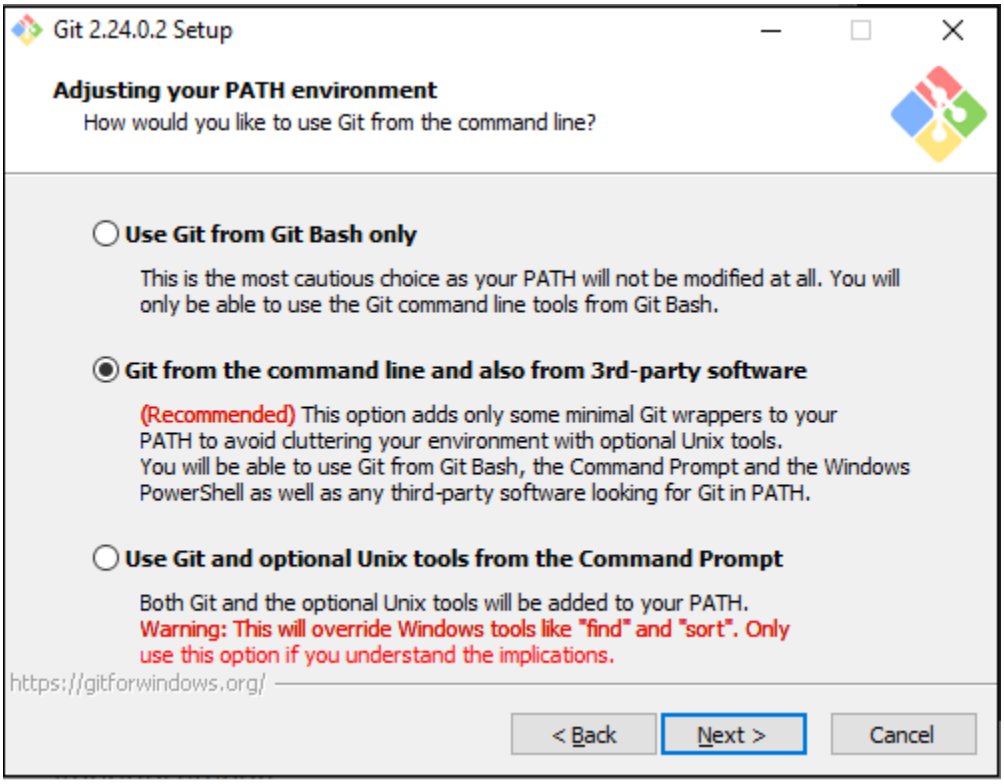
System Requirements

Operating System	Windows 7 or Later (I am Windows 10. You can also use Mac or Linux OS.).
Disk Space	400 MB (It does not include disk space for IDE/tools).
Tools	1. Windows PowerShell 2. Git for Windows 2.x (Here, Use Git from Windows Command Prompt option).
SDK	Flutter SDK for Windows
IDE	Android Studio (Official)

Install Git

Step 1: To download Git, https://git-scm.com/downloads/win.

Step 2: Run the .exe file to complete the installation. During installation, make sure that you have selected the recommended option.



Getting Started with Cross-Platform Mobile Application using Flutter

Flutter is an **open-source** mobile application development SDK created by **Google** to develop **cross-platform mobile applications**. Flutter makes it extremely easy and fast for even **novice programmers** (An computer programmers who is not experienced at any programming languages) .to build high-quality and responsive native mobile apps. No prior app development experience is required! .This article will show you how to build a simple **Hello Flutter** application and then run it on your own Android device!

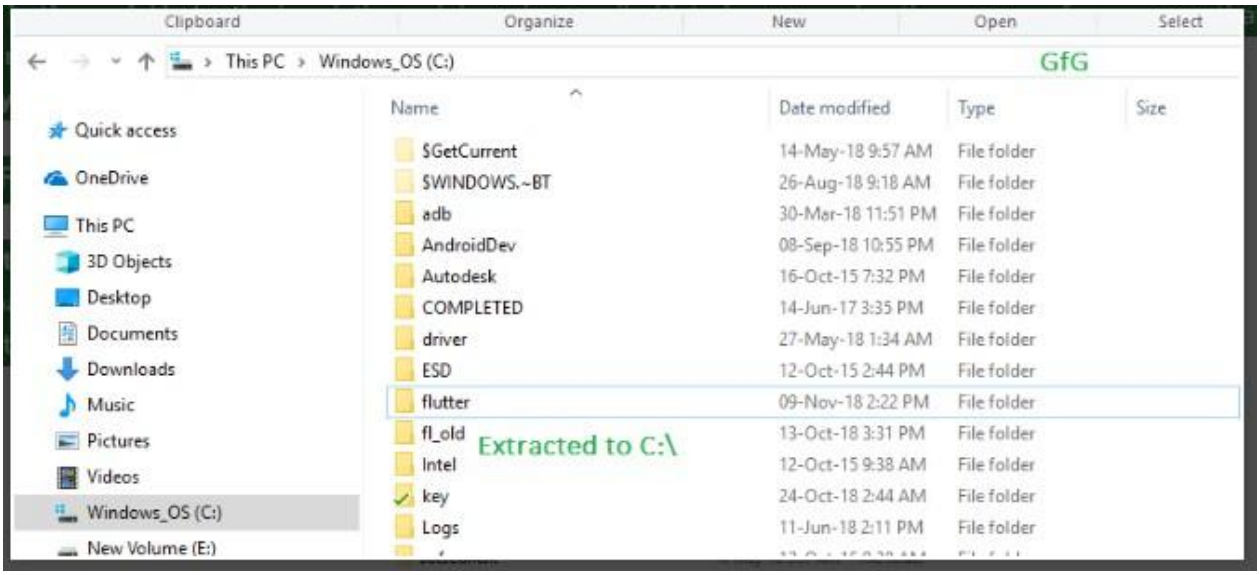
Why use Flutter over native Java development?

Flutter has some unique features compared to native Java and other SDK such as:

- **Fast Development**: Flutter makes use of customizable Widgets that can be nested together to create the app interface, similar to how HTML is structured. One can refer to this for details: Flutter for Web Devs.
- **Hot Reload**: Instantly see changes made in code, without recompiling.
- **It's Native in the end**: Under the hood, the codebase is compiled into Java, ensuring the same performance as that offered by native apps.
- **Light on system resources**: Flutter can be comfortably run on a lower resources system.

Setting Up Flutter

- **Step 1: Set up the Flutter SDK**
    1. Download the latest SDK here: Flutter SDK Archive
    2. Extract the zip file and place the contained '*flutter*' folder in the desired directory.



1. **Note**: It is recommended to not install Flutter in a directory which may require admin privileges, like 'C:\Program Files\'

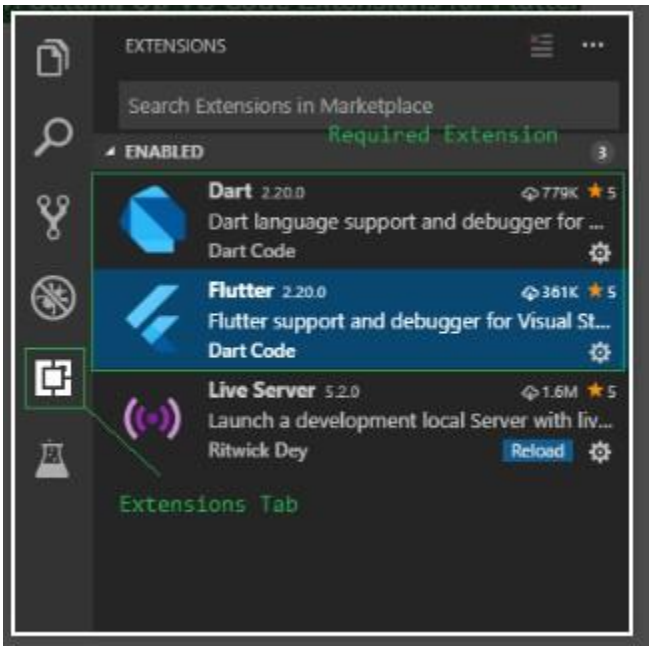
- **Step 2: Add Flutter to PATH**: Though not required, it is recommended to set the **PATH variable** to make Flutter easily accessible from anywhere on the system.
    1. Go to **Edit environment variables for your account** in Control Panel. You can search for this setting in your search bar.
    2. Under **User variables** check if there is an entry called PATH:
        - If it exists, add a new path to 'flutter\bin'.
        - If the entry does not exist, create a new entry named Path, and then add the full location to 'flutter\bin' (see example image)
    3. Reboot Windows after setting the PATH variable for it to work.

- **Step 3: Set up Android Studio**: Android Studio automatically downloads the development tools required for Flutter to work with Android.
    1. Download Android Studio here: Android Studio
    2. Start Android Studio and follow the SDK Manager wizard to download all the required build tools.

3. Set Up an Android Emulator
   - ✓ Open AVD Manager in Android Studio:
   - ✓ Go to Tools > Device Manager (or click the AVD icon in the toolbar).
   - ✓ Click Create Virtual Device.
   - ✓ Select a device from the list (e.g., Pixel 4) and click Next.
   - ✓ Download and select a system image (e.g., API Level 30).
   - ✓ Complete the setup and click Finish.
   - ✓ Start the emulator by clicking the Play button in the AVD Manager.

- **Step 4: Set up Visual Studio Code**: Visual Studio Code (or VS Code) is a light code editor that can be used in Flutter development. In this article, VS is used instead of Android Studio as it is lighter and has the minimal required features.

  1. Download and install VS Code: VS Code Download.
  2. To help in development, two VS Code extensions are recommended. Install the **Flutter** and **Dart** plugins from VS Code's Extension Tab. You can refer: Setting Up VS Code Extensions for Flutter

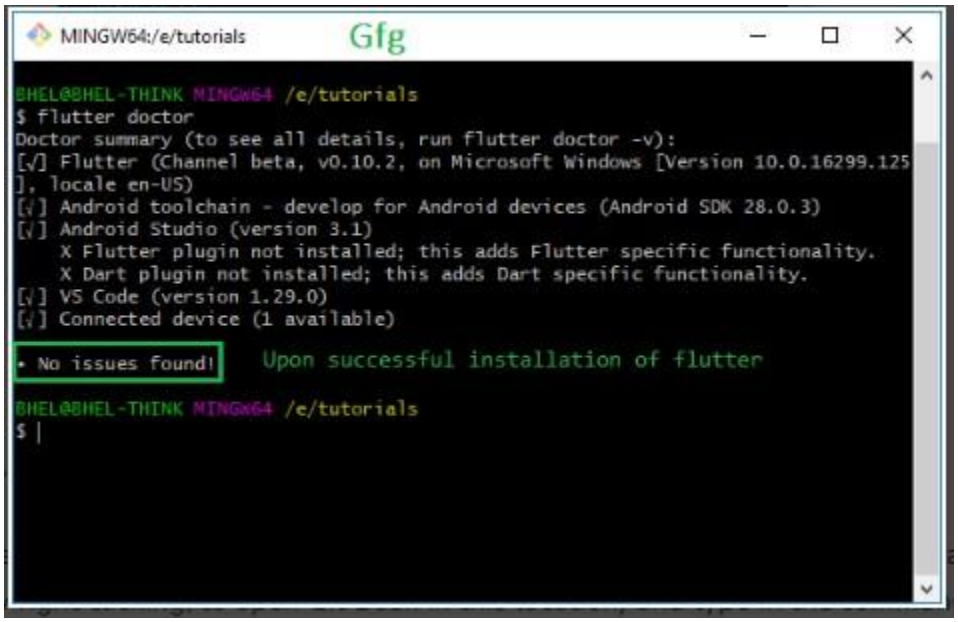


  3. **Install Git Bash**: Optional, but a recommended command prompt. Git Bash provides many common unix commands that are useful for some quick tasks.

  4. **Run Flutter Doctor**: Flutter doctor is an inbuilt tool by flutter that can be used to check the status of the Flutter installation. After setting up the PATH variable, you can open up the Command Prompt and execute:

```
flutter doctor
```

5. This will check everything required for Flutter to run and give a detailed report of any error it encounters.



<u>Install the Flutter and Dart plugins:</u>

After the successful installation of Android Studio, you have to install Flutter and Dart plugins. To do so follow the steps mentioned below:

1. **Start** Android Studio.
2. Open plugin preferences **(Configure > Plugins as of v3.6.3.0 or later)**.
3. Select the **Flutter** plugin and click Install.
4. Click Yes when prompted to install the **Dart** plugin.
5. Click **Restart** when prompted.

Open plugin preferences:

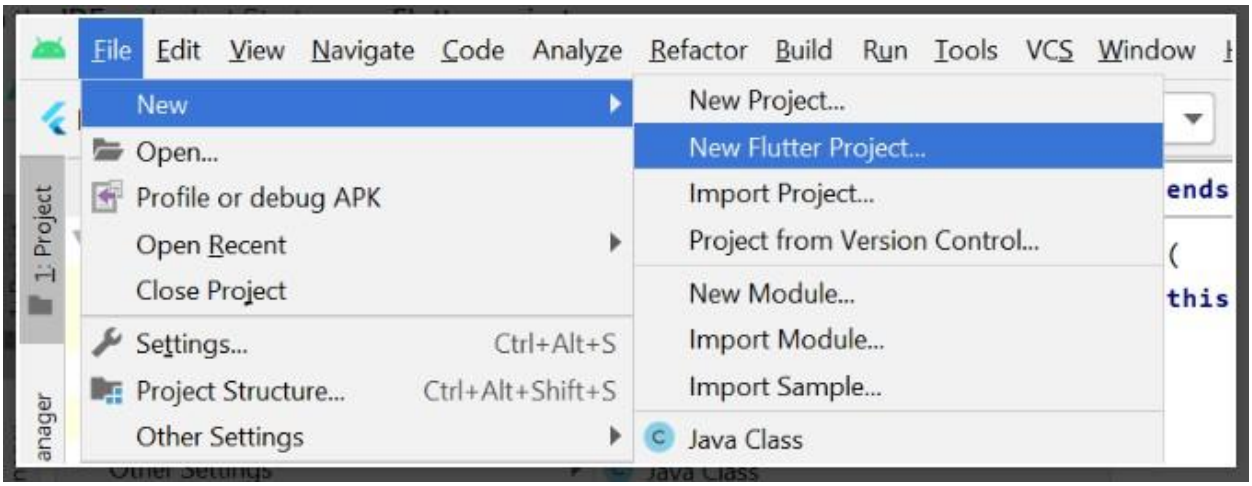
- For macOS: Preferences > Plugins on macOS,
- For Linux and Windows: File > Settings > Plugins

Now, Select Marketplace, select the Flutter plugin and click Install.

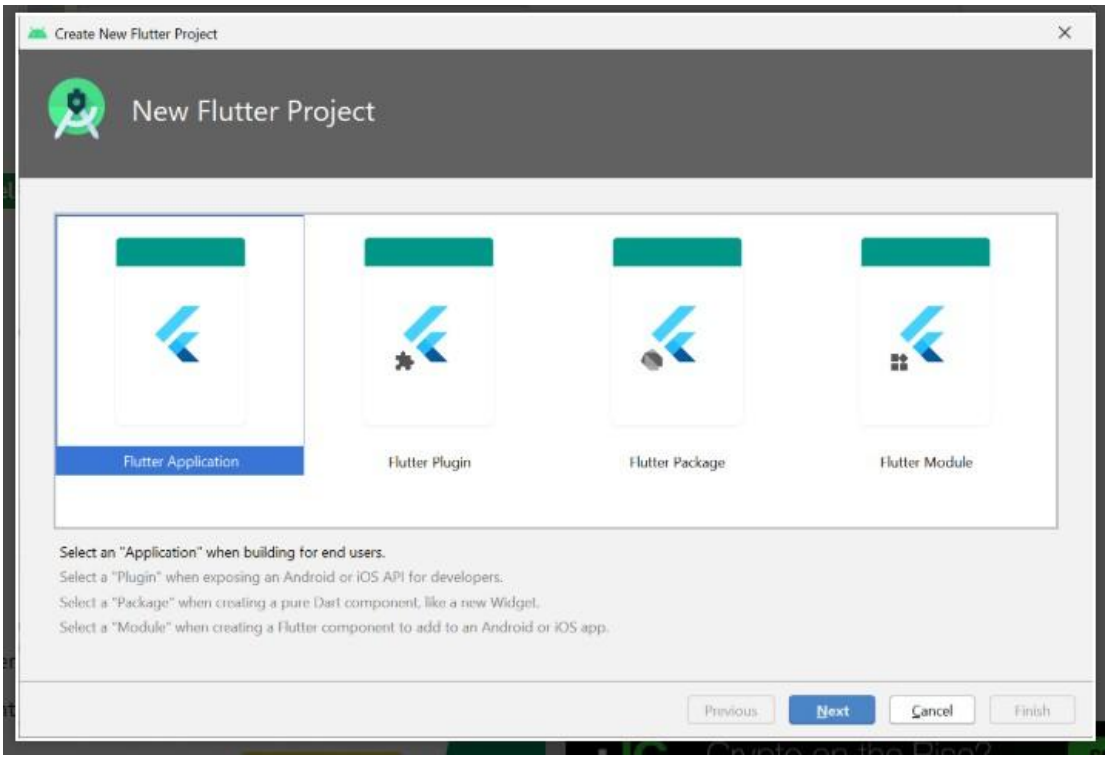
<u>Creating the application:</u>

After installing Dart and Flutter plugins create a flutter app to check if it is working properly or not, to do so follow the steps mentioned below:

Step 1: Open the **IDE** and select Start a **new Flutter project**.

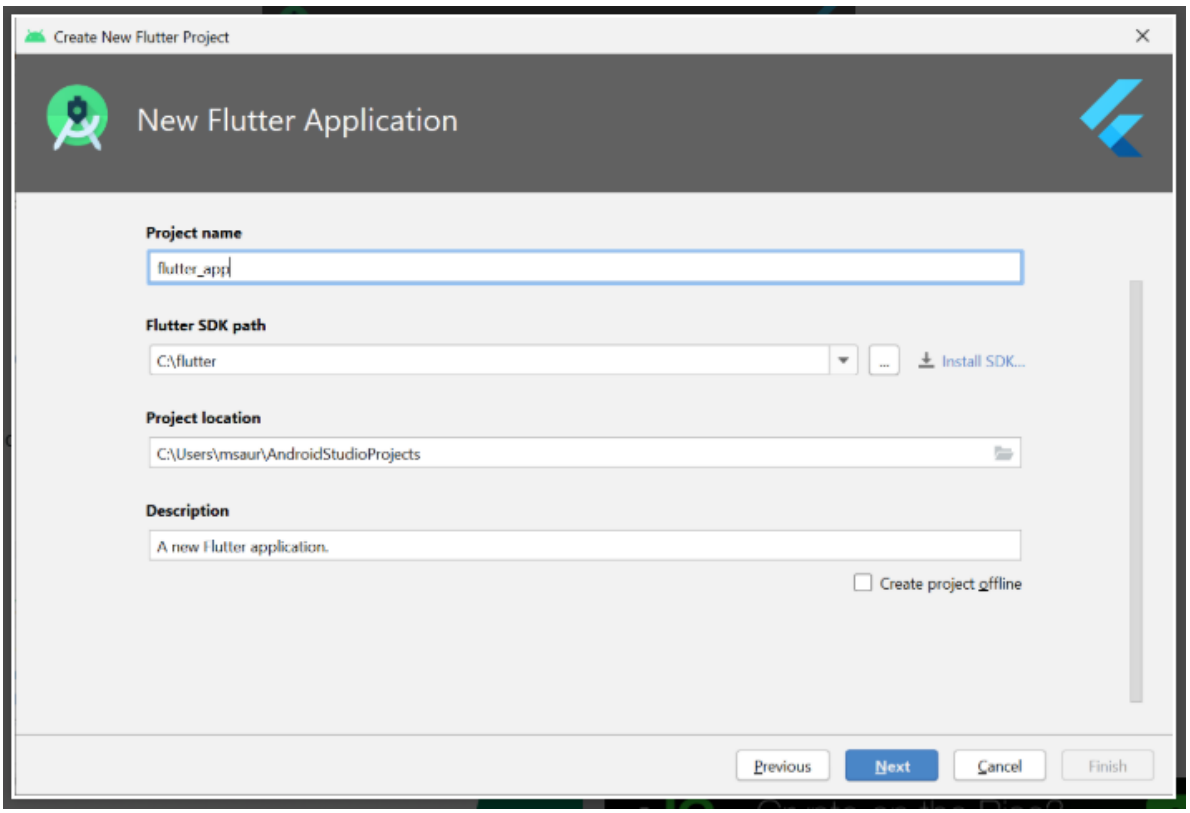


Step 2: Select the **Flutter Application** as the project type. Then click **Next**.

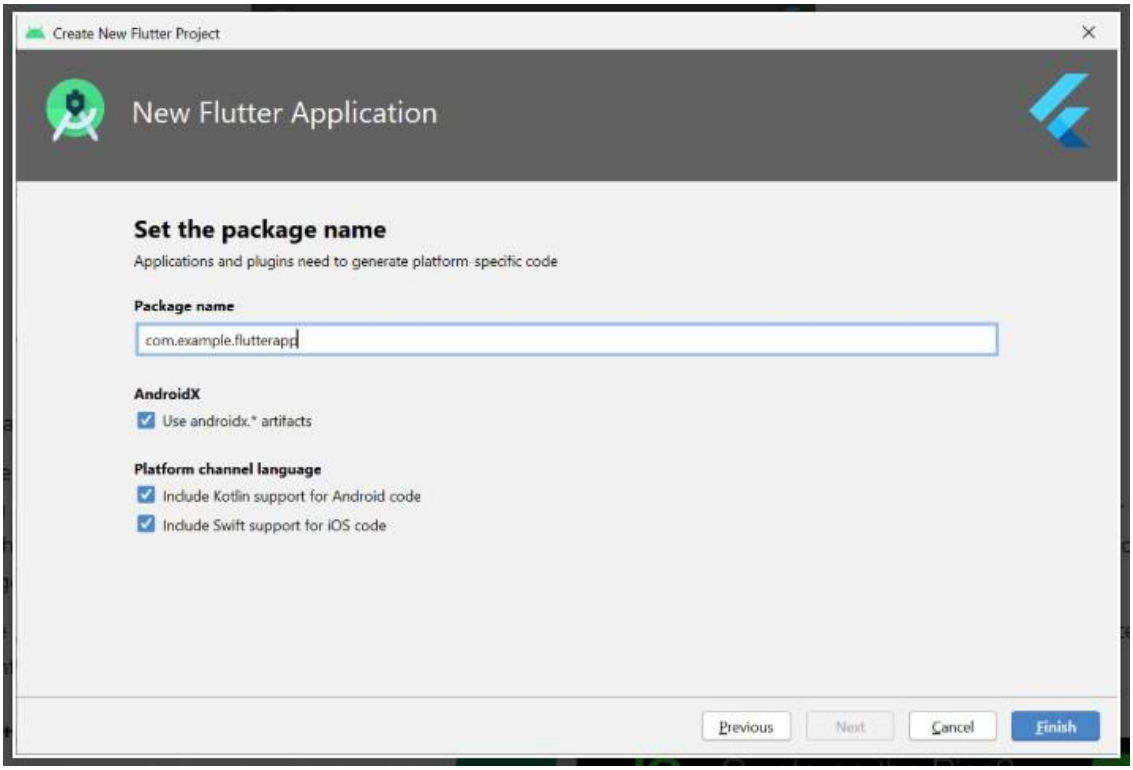


Step 3: Verify the **Flutter SDK path** specifies the SDK's location **(select Install SDK… if the text field is blank)**.

Step 4: Enter a project name (for example, *myapp*). Then click **Next**.



Step 5: Click **Finish**.



Step 6: Wait for Android Studio to install the SDK and create the project.

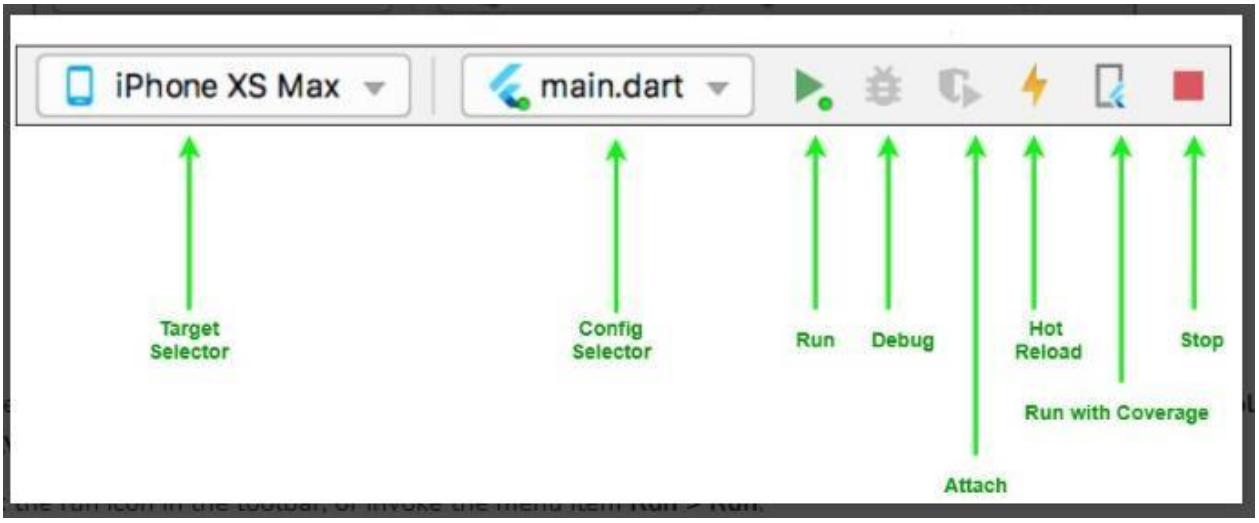
Note: When creating a new Flutter app, some Flutter IDE plugins ask for a company domain name in reverse order, something like com.example. The company domain name and project name are used together as the package name for Android (the Bundle ID for iOS) when the app is released. If you think that the app might be released, it's better to specify the package name now. The package name can't be changed once the app is released, so make the name unique.

The above steps create a Flutter project directory called *flutter_app* that contains a simple demo app that uses Material Components.

<u>Running the application:</u>

Follow the below steps to run the flutter application that was structured above:

Step 1: Locate the main Android Studio toolbar:



Step 2: In the **target selector**, select an Android device for running the app. If none are listed as available, select **Tools**> **Android** > **AVD Manager** and create one there.

Step 3: Click the run icon in the toolbar, or invoke the menu item **Run** > **Run**.

After the app build completes, you'll see the starter app on your device.